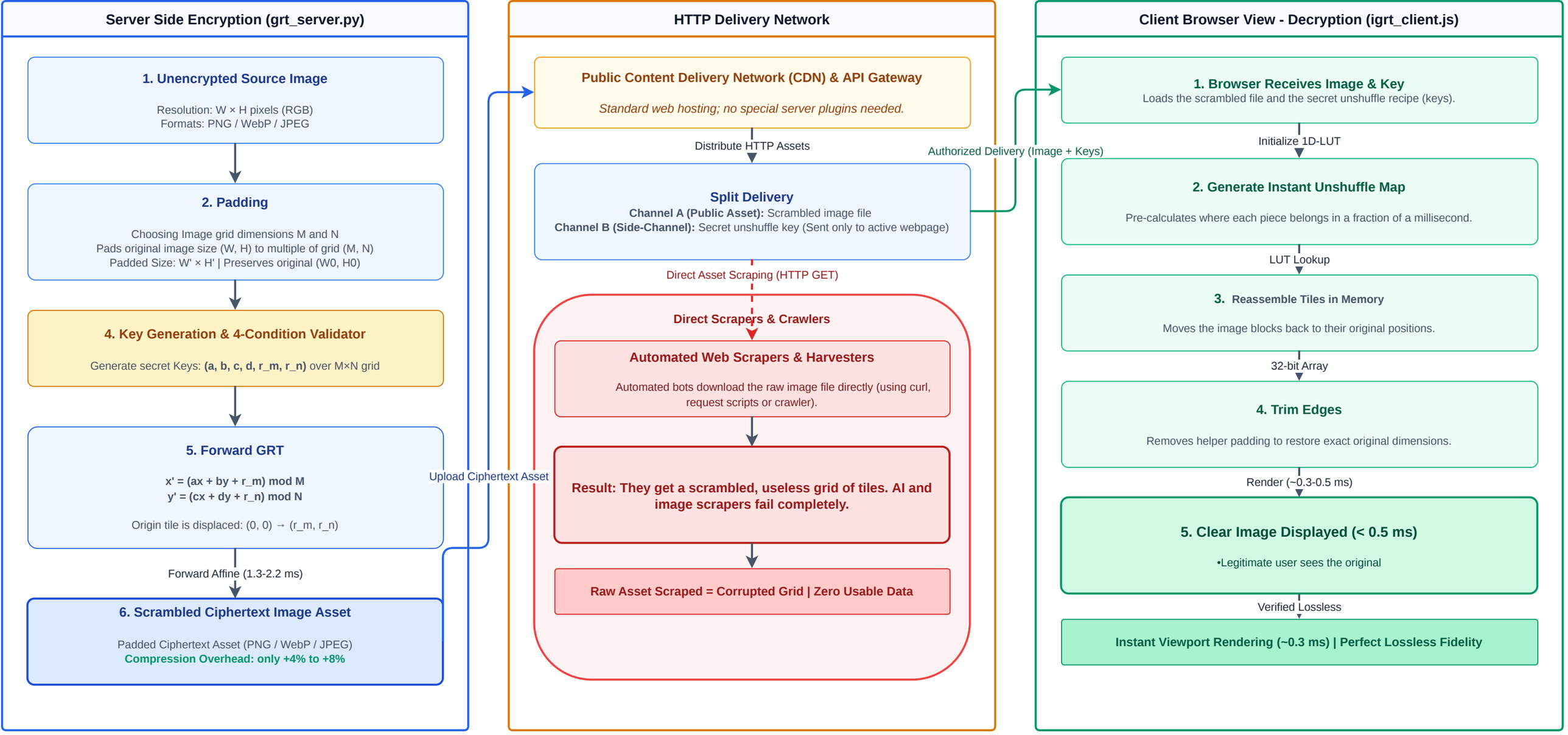

Server Side Encryption (grt_server.py)
1. Unencrypted Source Image
Resolution: W × H pixels (RGB)
Formats: PNG / WebP / JPEG
2. Padding
Choosing Image grid dimensions M and N
Pads original image size (W, H) to multiple of grid (M, N)
Padded Size: W' × H' | Preserves original (W0, H0)
4. Key Generation & 4-Condition Validator
Generate secret Keys: (a, b, c, d, r_m, r_n) over M×N grid
5. Forward GRT
x' = (ax + by + r_m) mod M
y' = (cx + dy + r_n) mod N
Origin tile is displaced: (0, 0) → (r_m, r_n)
Forward Affine (1.3-2.2 ms)
6. Scrambled Ciphertext Image Asset
Padded Ciphertext Asset (PNG / WebP / JPEG)
Compression Overhead: only +4% to +8%
Upload Ciphertext Asset
HTTP Delivery Network
Public Content Delivery Network (CDN) & API Gateway
Standard web hosting; no special server plugins needed.
Distribute HTTP Assets
Split Delivery
Channel A (Public Asset): Scrambled image file
Channel B (Side-Channel): Secret unshuffle key (Sent only to active webpage)
Direct Asset Scraping (HTTP GET)
Direct Scrapers & Crawlers
Automated Web Scrapers & Harvesters
Automated bots download the raw image file directly (using curl, request scripts or crawler).
Result: They get a scrambled, useless grid of tiles. AI and image scrapers fail completely.
Raw Asset Scraped = Corrupted Grid | Zero Usable Data
Authorized Delivery (Image + Keys)
Client Browser View - Decryption (igrt_client.js)
1. Browser Receives Image & Key
Loads the scrambled file and the secret unshuffle recipe (keys).
Initialize 1D-LUT
2. Generate Instant Unshuffle Map
Pre-calculates where each piece belongs in a fraction of a millisecond.
LUT Lookup
3. Reassemble Tiles in Memory
Moves the image blocks back to their original positions.
32-bit Array
4. Trim Edges
Removes helper padding to restore exact original dimensions.
Render (~0.3-0.5 ms)
5. Clear Image Displayed (< 0.5 ms)
•Legitimate user sees the original
Verified Lossless
Instant Viewport Rendering (~0.3 ms) | Perfect Lossless Fidelity